Concurrency & Distributed Systems

Week 3: Synchronization, Memory Semantics & Deadlocks
Day 2 — Deadlocks, Lock Ordering, and Prevention

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

Day 2 — From Correctness to Liveness Failure

Theme: From safe locking to structural deadlock risk

Today:

- What deadlock is
- Why it happens
- How opposite lock order creates it
- How global ordering prevents it
- How to think about debugging it

Goal

Understand deadlock as a **liveness failure**: the program is still alive, but progress has stopped.

Warm-Up — Spot the Deadlock

```
// Thread 1
synchronized(lockA) {
        synchronized(lockB) { }
}

// Thread 2
synchronized(lockB) {
        synchronized(lockA) { }
}
```

Question

Will this always deadlock? Sometimes? Never? Why?

Teaching purpose

Start with the puzzle first. Then let the theory explain it.

Block 1 — Deadlock Theory

Definition

Deadlock occurs when two or more threads are permanently waiting for each other's locks, so none of them can continue.

Key idea

Deadlock is not about wrong values. It is about **no progress**.

How Deadlock Feels in Practice

- The program may still be running
- CPU usage may drop very low
- No exception may be thrown
- Threads remain blocked forever

Observation

A deadlocked system is not terminated. It is trapped in a waiting structure it cannot escape.

Deadlock vs Starvation vs Livelock

Issue	Threads	Progress	Cause
Deadlock	Blocked	Never	Circular wait
Starvation	Runnable / waiting often	Never	Unfair scheduling
Livelock	Active	Never	Endless reactions without advance

Important

These are all liveness failures, but they are not the same problem.

Coffman Conditions for Deadlock

Deadlock requires four conditions

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Important

If one of these conditions is broken, deadlock cannot occur.

Quick Check — Which Conditions Can We Break?

Practical question

Which Coffman conditions can we realistically design against in Java?

- Mutual exclusion? Usually no — locks depend on it
- Hold and wait? Sometimes yes
- No preemption? Usually no for intrinsic monitors
- Circular wait? Yes — this is the main design lever

Takeaway

The most practical prevention strategy is often: **break circular wait**.

Condition 1 — Mutual Exclusion

Meaning

At least one resource must be held in an exclusive way.

- A Java monitor can be owned by only one thread at a time
- If a resource were freely shareable, deadlock would not arise on that resource

Interpretation

The same exclusivity that protects correctness can create deadlock risk.

Condition 2 — Hold and Wait

Meaning

A thread holds one lock while waiting to acquire another.

- Thread 1 holds lock A and waits for lock B
- Thread 2 holds lock B and waits for lock A

Interpretation

A thread is not releasing what it already owns before asking for more.

Condition 3 — No Preemption

Meaning

A lock cannot be forcibly taken away from a thread.

- Java does not let one thread steal another thread's monitor
- The owner must release the lock voluntarily

Interpretation

If locks could be revoked automatically, some deadlocks could be broken by force.

Condition 4 — Circular Wait

Meaning

A cycle exists in the waiting relationships.

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \cdots \rightarrow T_1$$

Analogy

Think of two cars facing each other on a narrow bridge. Each waits for the other to move first. Neither can progress.

The Most Important Condition to Watch

Practical teaching focus

All four conditions matter, but **circular wait** is the clearest structural pattern.

- Mutual exclusion is usually intentional
- No preemption is built into monitor behavior
- Hold-and-wait appears naturally in nested locking
- Circular wait is the pattern we can design against

$$\text{Prevent circular wait} \Rightarrow \text{prevent deadlock}$$

Block 2 — Creating Deadlock

Demo idea

Two locks are acquired in opposite order by two threads.

- Thread 1 acquires lock A, then tries for lock B
- Thread 2 acquires lock B, then tries for lock A

Expected result

The program freezes because each thread waits for the other forever.

Deadlock Demo — Thread 1

```
                synchronized(lockA) {
                        System.out.println("Thread 1 acquired lockA");

                        synchronized(lockB) {
                                System.out.println("Thread 1 acquired lockB");
                        }
                }
```

Order

Thread 1 acquires `lockA` first, then `lockB`.

Reentrancy note

Java locks are reentrant: a thread may re-acquire a lock it already holds. That prevents self-deadlock, but it does not prevent inter-thread cycles.

Deadlock Demo — Thread 2

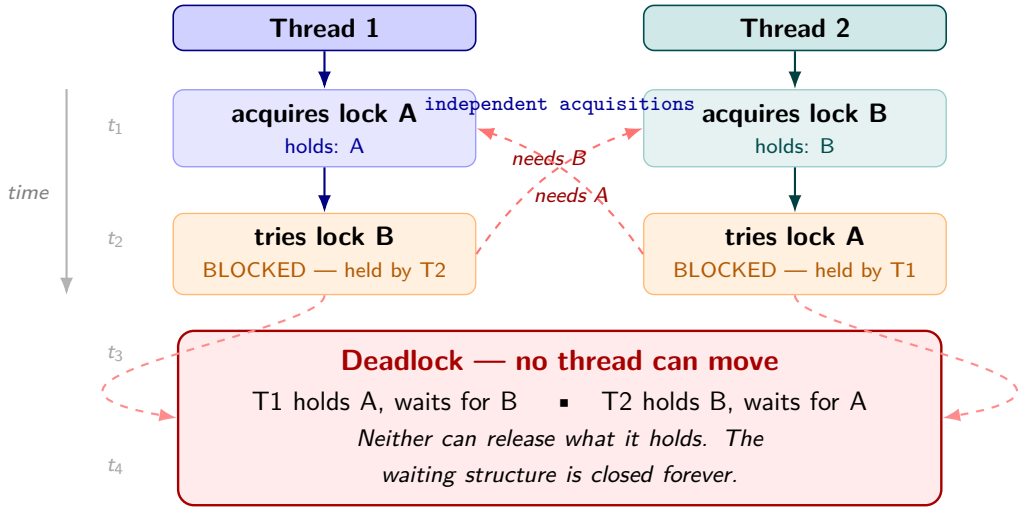
```
                synchronized(lockB) {
                        System.out.println("Thread 2 acquired lockB");

                        synchronized(lockA) {
                                System.out.println("Thread 2 acquired lockA");
                        }
                }
```

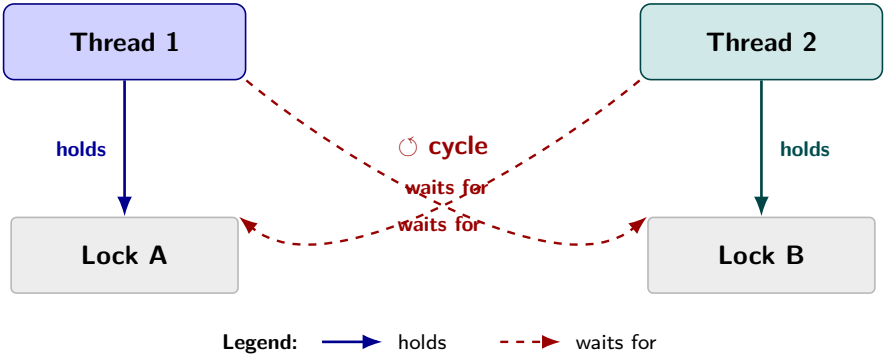
Order

Thread 2 acquires `lockB` first, then `lockA`.

Execution Trace — How Deadlock Forms



Deadlock — Resource Dependency Graph



The cycle is the deadlock

A cycle in the resource dependency graph means no thread in the cycle can ever proceed.

Remove the cycle — remove the deadlock.

Why It Never Progresses

1. Thread 1 cannot proceed until Thread 2 releases lock B
2. Thread 2 cannot proceed until Thread 1 releases lock A
3. Neither thread can release its first lock because each is blocked waiting for the second

Conclusion

No thread can move first. The waiting structure is closed.

Block 3 — Lock Ordering Principle

Rule

If all threads acquire locks in the same global order, deadlock cannot occur.

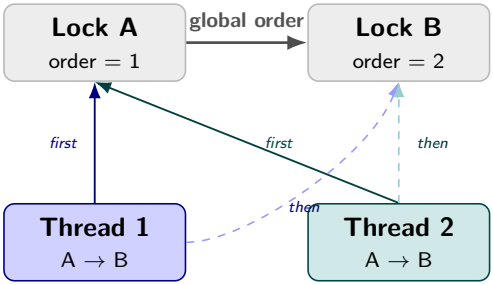
Core design idea

A consistent lock order destroys the possibility of circular wait.

Deadlock-Free Rewrite — Consistent Lock Order

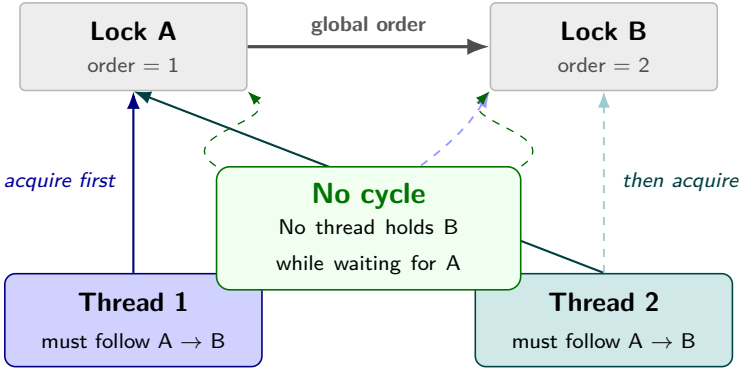
Both threads use the same order

```
// Thread 1
synchronized(lockA) {
        synchronized(lockB) {
                // critical section
        }
}
// Thread 2
synchronized(lockA) {
        synchronized(lockB) {
                        // critical
                            section
        }
}
```



No cycle
no thread holds B
while waiting for A

Why Lock Ordering Works



Key logic

If every thread acquires locks in the same global order, no thread can hold a higher-order lock while waiting for a lower-order one — the waiting graph **cannot contain a cycle**.

Lock Hierarchy

Concept

A **lock hierarchy** is a fixed global ordering of locks.

Example:

$$lockA < lockB < lockC$$

- Threads may acquire *lockA* then *lockB*
- Threads may acquire *lockB* then *lockC*
- Threads must never go backward in the hierarchy

Edge case

If locks are chosen dynamically at runtime, we need a consistent rule for ordering them, not just a visual hierarchy in the source code.

Dynamic Lock Ordering Example — Transfer

```
void transfer(Account from, Account to, int amount) {
        synchronized(from) {
                synchronized(to) {
                        // move money
                }
        }
}
```

Subtle bug

If one thread transfers A → B and another transfers B → A, the order is reversed dynamically.

How to Fix Dynamic Lock Ordering

Concrete strategy

Order locks by a unique, immutable criterion.

Examples:

- account ID
- object ID already assigned by design
- another stable global key

Example rule

Always lock the account with smaller ID first, then the larger one.

Deadlock Before vs After Ordering

Before	After
Opposite lock orders	One global lock order
Circular wait possible	Circular wait impossible
Program may freeze	Program progresses
Deadlock risk	Deadlock prevention by design

Main lesson

Deadlock prevention is often a design discipline, not a runtime repair.

Mini Exercise — Fix the Transfer

Challenge

Two threads may call:

$$transfer(A, B, \dots) \quad \text{and} \quad transfer(B, A, \dots)$$

at the same time.

Question

How do you synchronize this safely to avoid deadlock?

Hint

Do not lock in parameter order. Lock in a consistent global order.

Detecting Deadlocks in Practice

- Thread dumps with `jstack`
- VisualVM thread view
- JMX / `ThreadMXBean.findDeadlockedThreads()`

What to look for

In a thread dump, compare:

- `locked <0x...>`
- `waiting to lock <0x...>`

Matching monitor addresses reveal the cycle.

Inspection Preview

```
// Preview of a safer alternative with explicit locks:
if (lock.tryLock()) {
        try {
                // critical section
        } finally {
                lock.unlock();
        }
}
```

Bridge forward

If global ordering is impractical, later techniques include: `tryLock`, timeouts, and deadlock detection.

Summary of Day 2, Part 1

- Deadlock is a permanent waiting structure
- Coffman's four conditions explain when it is possible
- Circular wait is the most useful structural prevention target
- Opposite lock orders can create deadlock
- Global ordering prevents circular wait
- Real systems often need dynamic ordering rules

Next

Next we move from theory to inspection: thread dumps, `jstack`, VisualVM, and practical deadlock analysis.